

Pinout Reference — Wheeled-Leg Robot

0 · Peripheral → μ C Summary

Peripheral	μ C	Interface	Notes
BNO086 IMU	Teensy 4.1	SPI0 (pins 10–13, INT=2, RST=3)	9-DoF attitude
AK45-10 hip motors (L + R)	Teensy 4.1	CAN1 (pins 22/23)	MIT Cheetah protocol, IDs 11/12
ODrive wheel motors (axis 0/1)	Teensy 4.1	CAN2 (pins 0/1)	ODrive CAN v2, fw 0.5.6
FlySky receiver (iBUS)	Teensy 4.1	Serial4 RX (pin 16)	115200 baud, RX-only
AK45 UART	Teensy 4.1	Serial2 / Serial3	Half-duplex, pins 7–8 / 14–15
RGB status LED	Teensy 4.1	PWM (pins 4/5/6)	33 Ω series resistor
ESP32 link	Teensy 4.1 ↔ ESP32	Serial5 / UART2	1.2 Mbaud, pins 20/21 ↔ GPIO16/17
2.0" TFT display (ST7789)	ESP32	VSPI (GPIO18/23, CS=5, DC=2, RST=4, BLK=15)	240×320, adafruit ST7789 lib
VL53L1X ToF sensors (×4)	ESP32	I2C (GPIO21/22) + XSHUT (GPIO25–27, 32)	Addresses 0x30–0x33
NeoPixel strip (WS2812B, 30 LEDs)	ESP32	GPIO13 (FastLED)	5V power from buck; level-shift if needed
WiFi — telemetry / command / OTA	ESP32	Integrated radio	UDP 5005 / TCP 5006 / espota

1 · Teensy 4.1

Power

Stage	Device	Notes
24V → 5V	Buck converter (e.g. Pololu D24V50F5, or LM2596 module)	Must supply $\geq 1A$ for Teensy + peripherals

Stage	Device	Notes
5V → 3.3V	Teensy 4.1 built-in regulator (250 mA)	Powers Teensy logic, IMU, CAN transceivers
5V consumers	FlySky receiver, ESP32 DevKit V1 VIN pin, CAN transceiver VCC (if TJA1050)	

Teensy 4.1 is **3.3V logic** — **not 5V tolerant**. Use 3.3V CAN transceivers (SN65HVD230) or add level shifters.

Pin Assignments

Pin	Signal	Peripheral	Interface	Voltage	Notes
0	CAN2_RX	ODrive wheel motors	CAN2	3.3V	Via SN65HVD230 transceiver
1	CAN2_TX	ODrive wheel motors	CAN2	3.3V	Pins 0/1 = CAN2 only; Serial1 unavailable
2	IMU_INT	BNO086	SPI0	3.3V	Active-low interrupt
3	IMU_RST	BNO086	SPI0	3.3V	Active-low reset
4	LED_R	RGB LED	PWM	3.3V	33 Ω series resistor to LED
5	LED_G	RGB LED	PWM	3.3V	
6	LED_B	RGB LED	PWM	3.3V	
7	AK45_1_RX	AK45-1 UART	Serial2	3.3V	Half-duplex; use 1 kΩ series + diode
8	AK45_1_TX	AK45-1 UART	Serial2	3.3V	
9	(spare)	—	—	—	
10	IMU_CS	BNO086	SPI0	3.3V	Chip select, active low
11	SPI0_MOSI	BNO086	SPI0	3.3V	Hardware SPI bus
12	SPI0_MISO	BNO086	SPI0	3.3V	
13	SPI0_SCK	BNO086	SPI0	3.3V	Also drives built-in orange LED
14	AK45_2_TX	AK45-2 UART	Serial3	3.3V	
15	AK45_2_RX	AK45-2 UART	Serial3	3.3V	
16	IBUS_RX	FlySky receiver iBUS	Serial4 RX	3.3V*	See iBUS note below
17	(spare)	Serial4 TX unused	—	—	iBUS is RX-only
18	I2C_SDA	Spare / future sensor	Wire	3.3V	

Pin	Signal	Peripheral	Interface	Voltage	Notes
19	I2C_SCL	Spare / future sensor	Wire	3.3V	
20	EXT_MCU_TX	ESP32	Serial5	3.3V	Teensy TX → ESP32 GPIO16 (RX2)
21	EXT_MCU_RX	ESP32	Serial5	3.3V	Teensy RX ← ESP32 GPIO17 (TX2)
22	CAN1_TX	AK45-10 hip motors	CAN1	3.3V	Via SN65HVD230 transceiver
23	CAN1_RX	AK45-10 hip motors	CAN1	3.3V	Separate bus from ODrive
24	(spare)	—	—	—	
25	(spare)	—	—	—	
26	(spare)	—	—	—	
27	(spare)	—	—	—	
28–33	(spare)	—	—	—	Digital / analog / PWM

Bottom pads (solder only)

Pad	Signal	Notes
34	Serial8_RX	Spare UART
35	Serial8_TX	Spare UART
36–39	(spare)	
54–57	SDIO	microSD card (built-in slot on Teensy 4.1)

IMU SPI (BNO086)

BNO086 pin	Teensy pin	Notes
CS	10	Active low
MOSI (SDA)	11	
MISO (SA0)	12	
SCK (SCL)	13	
INT	2	Active low; input with pullup
RST	3	Active low; drive low 10 ms to reset
VCC	3.3V	

BNO086 pin	Teensy pin	Notes
GND	GND	
PS0	GND	PS0+PS1 both LOW = SPI mode
PS1	GND	

No pin collision with CAN — CAN1/CAN2 do not share any SPI0 pins on Teensy 4.1.

FlySky iBUS

- **Protocol:** 115200 baud, single-wire, receiver → controller only (RX pin only needed)
- **Signal voltage:** Most FlySky receivers (FS-iA6B, FS-X6B) output iBUS at **3.3V logic** when powered from 5V. Some older/clone units output 5V.
- **Safe wiring:** Add a **1 kΩ series resistor** between receiver iBUS pin and Teensy pin 16. This protects against a 5V unit without a full level shifter.
- **Library:** **IBusBM** (Arduino) — non-blocking, interrupt-driven, works on any Teensy Serial port.

2 · ESP32

Board: **esp32dev** (ESP32-WROOM-32, 3.3V logic, 5V tolerant VIN).

Teensy Link (Serial2 / UART2)

ESP32 GPIO	Signal	Connects to	Notes
GPIO16 (RX2)	UART2 RX	Teensy pin 20 (Serial5 TX)	
GPIO17 (TX2)	UART2 TX	Teensy pin 21 (Serial5 RX)	
VIN	Power in	5V rail (buck output)	
GND	Ground	Shared GND	

Baud rate: **1,200,000** (**TEENSY_UART_BAUD** in **esp32/src/config.h**). No level shifter needed — both sides are 3.3V logic.

2.0" TFT Display (GMT020-02-9P, ST7789, SPI)

VSPI bus — GPIO18/23 are hardware defaults shared across any SPI peripheral.

ESP32 GPIO	TFT Pin	Signal	Notes
GPIO18	SCL	SPI clock	VSPI CLK (hardware default)
GPIO23	SDA	SPI MOSI	VSPI MOSI (hardware default)
GPIO5	CS	Chip select	Active low

ESP32 GPIO	TFT Pin	Signal	Notes
GPIO2	DC	Data / command	
GPIO4	RST	Reset	Active low
GPIO15	BLK	Backlight	Drive HIGH to enable; or tie to 3.3V
3.3V	VCC	Power	
GND	GND	Ground	

Driver: ST7789 · Resolution: 240 × 320 · Library: [adafruit/Adafruit ST7735 and ST7789 Library](#) Pins defined in [esp32/src/config.h](#) as `TFT_CS` / `TFT_DC` / `TFT_RST` / `TFT_BLK`.

VL53L1X ToF Sensors (up to 4, I2C + XSHUT)

All sensors share the I2C bus. Each needs its own XSHUT pin so addresses can be assigned at boot. Default I2C address is 0x29; firmware reassigns each to 0x30–0x33 in sequence.

ESP32 GPIO	Signal	Sensor	Notes
GPIO21 (SDA)	I2C SDA	all	Wire I2C default
GPIO22 (SCL)	I2C SCL	all	Wire I2C default
GPIO25	XSHUT 0	sensor 0	→ address 0x30
GPIO26	XSHUT 1	sensor 1	→ address 0x31
GPIO27	XSHUT 2	sensor 2	→ address 0x32
GPIO32	XSHUT 3	sensor 3	→ address 0x33
3.3V	VCC	all	
GND	GND	all	

Library: [pololu/VL53L1X](#) · Env: [vl53l1x_demo](#) · Distance mode: Short (≤ 1.3 m) · Timing budget: 50 ms
Firmware: [esp32/src/vl53l1x/main.cpp](#)

Hello-world demo ([vl53l1x/main.cpp](#)) wires a single sensor with **XSHUT = GPIO19** (breadboard-accessible TX2-side pin). The production 4-sensor wiring above (GPIO25–27, 32) is the target pinout for the main firmware.

NeoPixel Strip (WS2812B)

ESP32 GPIO	Signal	Notes
GPIO13	NeoPixel DIN	FastLED WS2812B, GRB order
5V rail	VCC	Draw strip current from buck, not ESP32 3.3V reg
GND	GND	Shared GND

30 LEDs · 800 kHz · Animation task pinned to core 0 · Library: [fastled/FastLED](#) Firmware: [esp32/src/neopixel/main.cpp](#) Note: ESP32 outputs 3.3V logic; WS2812B spec minimum is 3.5V — add 74AHCT125 level shifter if flickering occurs.

WiFi Roles

Role	Protocol	Port
Telemetry stream (ESP32 → PC)	UDP	5005
Command link (PC → ESP32)	TCP	5006
OTA firmware update	espota	—

All WiFi I/O is handled by the ESP32-WROOM-32's integrated radio — no extra pins required.

3 · UARTs / CAN

Teensy–ESP32 UART Bridge

Teensy TX (pin 20)	→	ESP32 GPIO16 (RX2)
Teensy RX (pin 21)	←	ESP32 GPIO17 (TX2)
Teensy GND	→	ESP32 GND (shared ground required)
5V rail	→	ESP32 VIN

Serial Port Allocation Summary (Teensy 4.1)

Port	TX	RX	Assigned to
Serial1	1	0	UNAVAILABLE — pins used by CAN2
Serial2	8	7	AK45-1 UART
Serial3	14	15	AK45-2 UART
Serial4	17	16	FlySky iBUS RX
Serial5	20	21	ESP32 link
Serial6	24	25	Spare
Serial7	29	28	Spare
Serial8	35	34	Spare (bottom pads)

CAN Bus Architecture

Two separate CAN buses — cleaner isolation, no ID collision risk:

```

Teensy CAN1 (pins 22/23) → SN65HVD230 → CANH/CANL → AK45-10 L (ID 11 = 0x0B)
                                     → AK45-10 R (ID 12 = 0x0C)
                                     120 Ω terminator at each end

Teensy CAN2 (pins 0/ 1) → SN65HVD230 → CANH/CANL → ODrive axis 0 (node 0)
                                     → ODrive axis 1 (node 1)
                                     120 Ω terminator at each end

```

Each SN65HVD230 is powered from the 3.3V rail. No level shifting needed.

CAN ID Reference

CAN1 — AK45-10 Hip Motors (MIT Cheetah protocol)

Motor IDs are set in AK45 firmware (factory default 0x01; configured to **11 = left**, **12 = right** — decimal). All frames use the **same arbitration ID = motor ID**; response is distinguished by the first data byte equalling the motor ID.

ID (dec)	Direction	Frame content
11	TX → left hip	8-byte MIT control packet [p_des, v_des, Kp, Kd, t_ff]
11	RX ← left hip	8-byte status [motor_id, pos, vel, torque]
12	TX → right hip	same format
12	RX ← right hip	same format

Special command bytes (all other bytes 0xFF):

Byte 7	Command
0xFC	Enter MIT mode
0xFD	Exit MIT mode
0xFE	Set zero position

CAN2 — ODrive Wheel Motors (ODrive CAN v2, fw 0.5.6)

Arbitration ID format: (node_id << 5) | cmd_id ODrive axis 0 = node 0 (base 0x000), axis 1 = node 1 (base 0x020).

ID (axis 0)	ID (axis 1)	Dir	Command
0x001	0x021	RX ← ODrive	Heartbeat (axis state + error)
0x006	0x026	TX → ODrive	Clear errors

ID (axis 0)	ID (axis 1)	Dir	Command
0x007	0x027	TX → ODrive	Set axis state (e.g. CLOSED_LOOP = 8)
0x009	0x029	TX → (RTR)	Get encoder estimates → pos [turns], vel [turns/s]
0x00B	0x02B	TX → ODrive	Set controller mode
0x00D	0x02D	TX → ODrive	Set input velocity [turns/s] + torque_ff [Nm]
0x00E	0x02E	TX → ODrive	Set input torque [Nm]
0x014	0x034	TX → (RTR)	Get IQ [A]
0x018	0x038	TX → (RTR)	Get bus voltage & current